Multi-Country Accounts Payable Processing and Arabic Invoice AI Pipeline

Implementation Specification v1.0

SAP-OSS Arabic Corpus Working Group

April 2026

Abstract

This specification defines the full implementation of Standard Chartered Bank's multi-country Accounts Payable (AP) capability across the Middle East & Pakistan (MEP) footprint, with particular emphasis on the Arabic-language invoice processing pipeline sanctioned by the AI solutioning team under the initiative titled *Arabic Invoice, Peer analysis, & Translation* and announced by Manoj Hemath Kumar on 2025-11-23. Three orthogonal workstreams are specified in full:

- an **Arabic-invoice AI pipeline** that ingests PDF or e-mail attachments, performs bilingual OCR and schema-guided field extraction, applies an Arabic→English translation agent, and hands a validated record to downstream AP;
- a **multi-country VAT compliance engine** that enforces the regulator-specific checklists of ZATCA (Saudi Arabia), ETA (Egypt), NBR (Bahrain), and the Qatari counterparty conventions;
- an **AP workflow and approval engine** that formalises the seven activities of the Bahrain *Accounts Payable Departmental Operating Instruction* (DOI) as explicit state machines.

All three workstreams consume a single canonical data schema (Chapter 2) defined in `docs/arabic/structured/schema/`. The specification intentionally *excludes* the downstream SAP/PeopleSoft integration, which is covered by a separate interface document.

Reading order. Chapter 1 situates the work; Chapter 2 defines the normalised record types; Chapter 3 covers the AI pipeline; Chapter 4 covers the VAT engine; Chapter 5 covers the AP workflow engine; Chapter 6 specifies controls, testing, and acceptance; Chapter 7 is a traceability appendix linking every requirement to a source document and a structured record in `docs/arabic/structured/`.

Conventions. Arabic fragments are set with the `Geeza Pro` font family, for example هيئة الزكاة والضريبة والجمارك. File paths are typeset in `monospace` and are relative to the repository root. Enum values in running text appear in SMALL CAPS; they are defined authoritatively in `docs/arabic/structured/enums.yaml`.

Contents

Abstract **2**

1 Overview and scope **10**
1.1 The problem . . . 10
1.2 Goals . . . 10
1.3 In-scope countries and counterparties . . . 11
1.4 System context . . . 11
1.5 Out of scope . . . 11
1.6 Five-session engagement model . . . 12

2 Canonical data schema **13**
2.1 Layout of `structured/` . . . 13
2.2 Invoice record . . . 13
2.3 VAT compliance checklist . . . 15
2.4 Payment instruction . . . 15
2.5 Approval workflow . . . 16
2.6 DOI policy . . . 16
2.7 Enumerations . . . 16
2.8 Provenance . . . 17
2.9 Unified Schema Registry . . . 17

3 Arabic invoice AI pipeline **18**
3.1 Pipeline at a glance . . . 18
3.2 Stage 1 – Intake . . . 18
3.3 Stage 2 – OCR . . . 19
3.4 Stage 3 – Schema-guided field extraction . . . 19
3.5 Stage 4 – Arabic→English translation . . . 20
3.6 Stage 5 – Validation and routing . . . 21
3.7 Pipeline control plane . . . 22
3.8 Evaluation harness . . . 22

4 Multi-country VAT compliance engine **23**
4.1 Engine responsibilities . . . 23

4.2 Rule shape . . . 23
4.3 Saudi Arabia (ZATCA) . . . 24
4.3.1 Rules . . . 24
4.3.2 Decision function . . . 24
4.3.3 Worked example . . . 25
4.4 Egypt (ETA) . . . 25
4.4.1 Rules . . . 25
4.4.2 Decision function . . . 25
4.4.3 Worked examples . . . 26
4.5 Bahrain (NBR) . . . 26
4.5.1 Rules . . . 26
4.5.2 BHD 500 threshold . . . 28
4.6 Qatar (QTA) . . . 28
4.7 Withholding tax . . . 28
4.8 Engine outputs . . . 28

5 AP workflow and approval engine **29**
5.1 Engine responsibilities . . . 29
5.2 Activity view of the Bahrain DOI . . . 29
5.3 Workflow selection . . . 29
5.4 `po-eproc` state machine . . . 29
5.5 `manual-one-off` state machine . . . 31
5.6 `manual-recurring` state machine . . . 31
5.7 `vat-payment` state machine . . . 31
5.8 Delegation-of-authority . . . 31
5.9 Exceptions . . . 32
5.10 Payment-suspense reconciliation . . . 32
5.11 Auditability . . . 32

6 Controls, testing, and acceptance **33**
6.1 Control catalogue . . . 33
6.2 Testing strategy . . . 33
6.3 Regression golden set . . . 34
6.4 Acceptance criteria per workstream . . . 35
6.5 Traceability matrix . . . 35
6.6 Operational readiness . . . 35
6.7 Deployment and rollback . . . 36
6.8 Audit event log . . . 37

7 References **38**
7.1 Source documents . . . 38
7.2 Derived text . . . 38

7.3 Structured records . . . 39
7.4 Tools . . . 39
7.5 Minimal corpus excerpt . . . 39
7.6 Out-of-scope assets . . . 41

8 VAT Module Implementation Specification 42
8.1 Module architecture . . . 42
8.2 API contracts . . . 43
8.2.1 Registration validation . . . 43
8.2.2 VAT calculation . . . 43
8.2.3 Activity lookup . . . 44
8.3 JSON-Logic expression specification . . . 44
8.3.1 Supported operators . . . 44
8.3.2 Variable paths . . . 44
8.3.3 Named helper invocation . . . 45
8.4 Error handling specification . . . 45
8.4.1 Error codes . . . 45
8.4.2 Fallback behaviour . . . 45
8.5 Performance requirements . . . 46
8.6 Implementation checklist . . . 46

9 Testing Specification 47
9.1 Test coverage requirements . . . 47
9.2 Test fixtures . . . 47
9.2.1 Invoice fixtures . . . 47
9.2.2 OCR fixtures . . . 47
9.3 Unit test cases . . . 48
9.3.1 Invoice extraction . . . 48
9.3.2 VAT validation . . . 48
9.4 Integration test suite . . . 48
9.4.1 End-to-end scenarios . . . 49
9.5 Performance benchmarks . . . 49
9.5.1 Reference hardware profile . . . 49
9.6 Security tests . . . 50
9.6.1 Prompt injection . . . 50
9.7 CI/CD integration . . . 50
9.8 Acceptance criteria . . . 50

10 OCR and Translation Calibration Pilot Design 51
10.1 Objective . . . 51
10.2 Pilot design . . . 51
10.2.1 Sample size . . . 51

10.2.2 Document selection . . . 52
10.2.3 Ground truth creation . . . 52
10.3 Threshold calibration methodology . . . 52
10.3.1 OCR evaluation . . . 52
10.3.2 Threshold determination . . . 53
10.3.3 Translation quality gate calibration . . . 54
10.3.4 Confidence interval calculation . . . 54
10.4 Success criteria . . . 54
10.5 Fallback procedure . . . 54
10.6 Timeline . . . 55
10.7 Ownership . . . 55
10.8 Post-pilot monitoring . . . 55

11 Project Roadmap **56**
Scope of this chapter . . . 56
11.1 MoSCoW classification . . . 56
11.2 Dependency graph . . . 59
11.3 Project milestones . . . 59
11.4 Launch tiers and exit gates . . . 59
11.5 Kill-switch conditions . . . 59
11.6 Shared-platform dependencies . . . 59
11.7 v2.0 expansion trigger . . . 63
11.8 Governance and business-case cadence . . . 63

List of Tables

1.1 Countries in scope, with the controlling source document and the structured records derived from it. 11
1.2 Five-session engagement model (codified in `docs/arabic/structured/enums.yaml`). 12

2.1 Directory layout under `docs/arabic/structured/`. 13
2.2 Top-level fields of `invoice.schema.json`. 15
2.3 Fields of a VAT checklist record. 15

3.1 Extraction pitfalls observed in the corpus and mitigations. 20
3.2 Locked glossary. The right-hand column is what the translation agent *must* output. 21

4.1 KSA VAT rules derived from the Q3 2025 OTP checklist. 24
4.2 ETA e-invoice rules. 25
4.3 NBR VAT invoice types recognised in Bahrain AP. Each row maps to a `vat_category` enum value. 26

5.1 The seven DOI activities and the workflow paths they drive. 30
5.2 Default DOA thresholds per workflow. 32

6.1 Control catalogue and enforcement mechanisms. 33
6.2 Traceability of specification to source document and structured record. 36

8.1 VAT registration number formats by country. 43
8.2 Activity code to VAT treatment mapping (Egypt, partial). 44
8.3 JSON-Logic operators supported by the rules engine. 45
8.4 VAT module error codes. 46
8.5 VAT module performance requirements. 46

9.1 Arabic AP test coverage targets. 47
9.2 Required invoice fixture categories. 47
9.3 Arabic AP performance targets on reference hardware. 49

10.1 Pilot sample size by country. 51
10.2 OCR and translation pilot timeline. 55

10.3 Pilot roles and responsibilities. 55

11.1 MoSCoW prioritisation (shared across the four SAP OSS Finance projects). . 57
11.2 Arabic AP v1.0 project milestones in dependency order. Priority: M=Must (critical path), S=Should, C=Could, W=Won't. 58
11.3 Arabic AP launch tiers. The exit gate defines the MoSCoW items that must be closed to exit the tier. 60
11.4 Arabic AP kill-switch. 61
11.5 Shared-platform dependencies. 62

List of Figures

1.1 System context. Regulator portals (blue) and vendor PDFs feed the AI pipeline (orange), which emits canonical records to the deterministic VAT and approval engines (green). Downstream posting to GL is out of scope. . . 11

3.1 The five pipeline stages; each emits a well-typed payload to the next. Only Stage 5 writes to the canonical corpus. 18
3.2 Pipeline control-plane states for a single invoice. 22

4.1 KSA decision function. The first matching branch determines the emitted `tax_type_classification`. 24
4.2 Bahrain NBR classification cascade. The first branch whose predicate holds emits the `vat_category` and short-circuits the remaining tests. 27

5.1 Workflow selection decision tree applied at Stage 5 of the AI pipeline (Chapter 3). 30
5.2 `po-eproc` state machine (`docs/arabic/structured/workflows/po-eproc.json`). 30
5.3 `manual-one-off` (`docs/arabic/structured/workflows/manual-one-off.json`). 31
5.4 `vat-payment`: linear path walked by the ZATCA Q3 2025 assessment. Entry requires every VAT checklist rule to resolve to Y or NA. 31

6.1 Control enforcement points along the end-to-end invoice lifecycle. Dashed arrows show where each DOI control (Table 6.1) is evaluated. 34

8.1 VAT module component diagram. Dashed arrows indicate helper invocations from the rules engine. 42

11.1 Arabic AP dependency graph. Red nodes are Must (critical path); blue are Should (country-wave allowance); dotted green are Could (discretionary). Solid arrows are hard prerequisites; dashed arrows indicate conditional starts (the target is unlocked when the upstream completes *and* the multilingual volume trigger in Section 11.7 fires). 59

Chapter 1

Overview and scope

1.1 The problem

Standard Chartered Bank operates Accounts Payable across a heterogeneous regulatory and linguistic landscape. Depending on the country, an incoming invoice may be:

- issued by a national e-invoicing portal (Egypt's ETA, with a 26-character ELECTRONIC_ID; Saudi Arabia's ZATCA with an analogous assessment reference);
- purely Arabic (as in the two Egyptian invoices `3CPJR5CQ…K10.pdf` and `B1STE8RH…SJ10.pdf` and the Saudi VAT assessment اشعار صدور فاتورة);
- bilingual (Qatar's QCSD / EDAA memo);
- wholly English with pre-assigned GL coding (e.g. the Bahrain DOI-covered PO flow).

Today the Arabic-language material is handled manually: a finance analyst reads the PDF, transcribes key fields into an OTP/VAT checklist (see the Saudi Q3 2025 checklist in `docs/arabic/source/Saudi…OTP…`), and routes it through the DOI approval chain. The November 2025 e-mail thread from Manoj Hemath Kumar identifies this manual path as the target for AI replacement.

1.2 Goals

The specification targets three outcomes, each with a direct lineage to the sample corpus in `docs/arabic/source/`:

1. **Structured ingestion.** Every Arabic or bilingual invoice becomes a record conforming to `structured/schema/invoice.schema.json` (Chapter 2).
2. **Deterministic compliance.** Every invoice is passed through a rule engine that emits a VAT compliance checklist conforming to `structured/schema/vat-checklist.schema.json` (Chapter 4), with the rule set derived from the regulator-specific controls in the corpus.

3. **Auditable approval.** Every payment follows one of the four state machines in `docs/arabic/structured/workflows/` and meets the controls enumerated in the Bahrain DOI (Chapters 5 and 6).

1.3 In-scope countries and counterparties

Table 1.1: Countries in scope, with the controlling source document and the structured records derived from it.

Country	Authority	Currency	Source document(s)
Egypt (EG)	ETA	EGP	`3CPJR5CQ…K10.pdf`, `B1STE8RH…SJ10.pdf`
Saudi Arabia (SA)	ZATCA	SAR	اشعار صدور فاتورة (005).pdf, Saudi OTP/VAT Q3'25 checklist
Qatar (QA)	QTA	QAR	EDAA Annual Fee 2025, QCSD payment instruction
Bahrain (BH)	NBR	BHD	Accounts Payable DOI
Iraq (IQ), UAE, PK	various	—	*Planned*; referenced in the initiative e-mail but formally out of scope until at least five sample invoices per jurisdiction are added to `source/`

1.4 System context

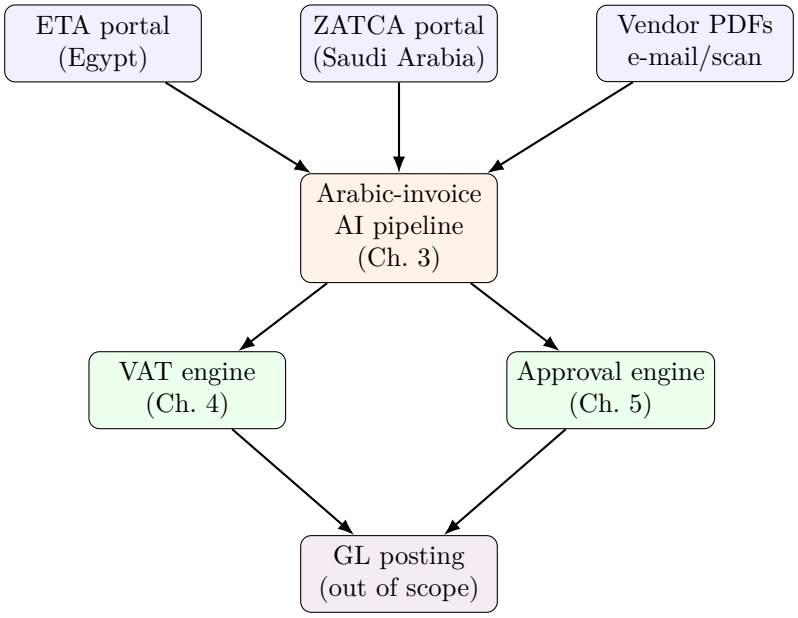

Figure 1.1: System context. Regulator portals (blue) and vendor PDFs feed the AI pipeline (orange), which emits canonical records to the deterministic VAT and approval engines (green). Downstream posting to GL is out of scope.

1.5 Out of scope

The following are *explicitly* out of scope for this document:

- Integration contracts with PeopleSoft PSAP/PSGL, eBBS, eOPS, and Tax-Q. These are covered by a separate interface specification.
- Vendor master-data on-boarding and the 48-hour screening service itself; this document assumes a screening result is available as a boolean precondition.
- The encrypted template `Business Instruction Template - V1.0 (002) (1).xlsm` and the duplicate `EDAA…[65].pdf`; neither is referenced by any structured record.
- Peer/competitor analysis across jurisdictions, which the e-mail thread itself identifies as a separate initiative.

1.6 Five-session engagement model

Per the e-mail, every country instance of this system follows a five-session agile discovery cadence (Table 1.2); the AI pipeline of Chapter 3 is designed to advance with that cadence.

Table 1.2: Five-session engagement model (codified in `docs/arabic/structured/enums.yaml`).

Code	Name	Goal
UNDERSTAND	Session 1	Map problem, define user journey, pick target.
SKETCH	Session 2	Develop detailed solution design.
DECIDE	Session 3	Critique approaches, select best path.
PROTOTYPE	Session 4	Build practical system for use case.
TEST	Session 5	Test with $\geq$5 stakeholders, gather feedback.

Chapter 2

Canonical data schema

2.1 Layout of `structured/`

The machine-readable corpus is organised as in Table 2.1. All records validate against a JSON Schema Draft 07 file under `docs/arabic/structured/schema/`. The `docs/arabic/structured/validate.py` script walks the tree, resolves `$ref` entries via a local registry (so offline validation succeeds), and exits non-zero on any failure; it is the authoritative acceptance gate for this chapter.

Table 2.1: Directory layout under `docs/arabic/structured/`.

Path	**Contents**
`schema/invoice.schema.json`	Canonical invoice / assessment record.
`schema/vat-checklist.schema.json`	Regulator checklist (KSA 11-point; extensible).
`schema/payment-instruction.schema.json`	Vendor payment memo.
`schema/approval-workflow.schema.json`	Declarative state machine.
`schema/doi-policy.schema.json`	DOI with activities and controls.
`invoices/*.json`	Invoice records (EG ETA $\times 2$; SA ZATCA $\times 1$).
`payment-instructions/*.json`	Qatar QCSD/EDAA records.
`vat-checklists/*.json`	Saudi OTP/VAT Q3 2025.
`policies/*.json`	Bahrain AP DOI.
`workflows/*.json`	PO-eProc, manual-one-off, manual-recurring, VAT payment.
`enums.yaml`	Controlled vocabulary.
`corpus.json`	Flat index of every source $\leftrightarrow$ text $\leftrightarrow$ record link.
`validate.py`	Offline validator.

2.2 Invoice record

The invoice record is the central object shared by ingestion (Chapter 3), the VAT engine (Chapter 4), and the approval engine (Chapter 5). Table 2.2 enumerates the top-level fields; Listing 2.1 is a worked example taken verbatim from `docs/arabic/structured/invoices/eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10.json`.

Listing 2.1: Full invoice record for the Egypt ETA seal invoice (`eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10`). Arabic strings are UTF-8; the LaTeX engine renders them RTL.


```
{
  "id": "eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10",
  "schema_ref": "invoice.schema.json",
  "metadata": {
    "country": "EG",
    "tax_authority": "ETA",
    "document_kind": "E_INVOICE",
    "electronic_id": "3CPJR5CQ4PNNN9A4XHBB370K10",
    "issue_date": "2025-07-15",
    "submission_date": "2025-07-15T16:10:00+00:00",
    "language": "ar",
    "status": "VALID"
  },
  "seller": {
    "name_ar": "            ",
    "tax_id": "438212282",
    "tax_id_type": "ETA_TRN",
    "activity_code": "1811"
  },
  "buyer": {
    "name_ar": "            ",
    "tax_id": "670307459",
    "tax_id_type": "ETA_TRN"
  },
  "line_items": [{
    "description_ar": "           42       ",
    "description_en": "Circular Trodat seal 42mm, series",
    "quantity": 1, "unit_of_measure": "EA",
    "unit_price": 580000.00, "line_total": 580000.00
  }],
  "totals": {
    "currency": "EGP",
    "net": 580000.00, "vat_amount": 81200.00,
    "vat_rate_percent": 14, "vat_category": "FI",
    "gross": 661200.00
  },
  "source_document": {
    "path": "docs/arabic/source/3CPJR5CQ4PNNN9A4XHBB370K10.pdf",
    "sha256": "2b9126...4b2c0",
    "ocr_text_path": "docs/arabic/extracted-text/eg-eta-3CPJR...txt"
  }
}
```

Table 2.2: Top-level fields of `invoice.schema.json`.

Field	Type	Req.	Notes
`id`	string	✓	Filename stem under `invoices/`.
`metadata`	object	✓	Country, authority, dates, language, status.
`seller`	party	✓	Includes Arabic name, English name, `tax_id_type`.
`buyer`	party	✓	Same shape as seller.
`line_items[]`	array	–	At least one of `description`, `description_ar`, `description_en`.
`totals`	object	✓	`currency`, `gross` required.
`tax_period`	object	–	Populated for periodic assessments (ZATCA).
`signature`	object	–	Kind ∈ {SEAL, WET_INK, DIGITAL, SYSTEM_GENERATED}.
`source_document`	object	✓	`path`, `sha256`, `ocr_text_path`.
`notes[]`	array	–	Free-form analyst notes.

Invoice ELECTRONIC_ID for ETA must match the regex in `docs/arabic/structured/enums.yaml` (`^[0-9A-Z]{26}$`); the ZATCA assessment number is a decimal prefix matching the taxpayer's ZATCA_TDN. The `status` enum is {VALID, INVALID, PENDING, CANCELLED}; ZATCA assessments are always VALID on arrival.

2.3 VAT compliance checklist

The KSA OTP/VAT checklist is encoded as an ordered array of items (Table 2.3). Each item records the regulator's question, the analyst's verdict (Y/N/NA), and the extracted value. The authoritative list is the eleven points in the Saudi Q3 2025 checklist; NBR (Bahrain) maintains an analogous set drawn from Activity 3 of the DOI.

Table 2.3: Fields of a VAT checklist record.

Field	Type	Req.	Notes
`period`	object	✓	`start`, `end`, `label`.
`checklist[]`	array	✓	Items indexed 1…n; each has `result` ∈ {Y,N,NA}.
`gl_coding`	object	✓	Chart fields (account/product/dept/OU/class).
`supplier`	object	✓	`supplier_type` ∈ {GOVERNMENT, VENDOR, EMPLOYEE, INTER_BU}.
`tax_type_classification`	string	✓	VAT category enum.
`withholding_tax`	object	–	Rate and amount if applicable.
`psid`	string	–	Preparer PSID.

2.4 Payment instruction

Derived from the Qatar QCSD instruction. The approval chain is an array of typed signatories; each entry carries PSID, role, and approval timestamp — giving a tamper-evident audit trail once hashed. The `chart_field` is stored as a dash-separated composite string (e.g. `665011-4551-800` = account-product-operating_unit) because this is the convention used by the source memos.

Listing 2.2: Excerpt from `structured/vat-checklists/sa-otp-vat-q3-2025.json` showing items 1–5 of the KSA 11-point checklist.

```
"checklist": [
  {"n": 1, "label": "Invoice in Arabic language",                    "result": "Y"},
  {"n": 2, "label": "Addressed to Standard Chartered Capital",       "result": "Y"},
  {"n": 3, "label": "Date of issue",                                 "result": "Y", "value"
    : "2025-10-30"},
  {"n": 4, "label": "Sequential number",                             "result": "Y", "value"
    : "3003093648253002"},
  {"n": 5, "label": "Tax Identification Number (15-digit)",          "result": "NA",
     "regulation": "Supplier is ZATCA itself -- government body"}
],
```

2.5 Approval workflow

Each workflow record is a deterministic state machine. States are nodes; transitions are edges tagged with a trigger and, where relevant, a `required_role` and a list of `preconditions`. The `delegation_of_authority` array gates roles by LCY threshold and is consulted by the approval engine before any transition out of a "x-approved" state. Four workflow instances are shipped in the corpus:

- `po-eproc` — PO / eProcurement path (DOI Activity 3);
- `manual-one-off` — one-off manual payment (DOI Activity 5);
- `manual-recurring` — recurring manual with dispensation (DOI Activity 5);
- `vat-payment` — authority-counterparty (e.g. ZATCA) path.

2.6 DOI policy

The Bahrain Accounts Payable DOI is represented as a `doi-policy` record whose `activities[]` array preserves the original numbering (1–7) and whose `controls[]` array gives each control an ID, description, pointer back to the activity, and the risk it mitigates. This is what the approval engine consults to decide *which* workflow instance applies to a given invoice.

2.7 Enumerations

The file `docs/arabic/structured/enums.yaml` is the single source of truth for controlled vocabulary used across the corpus: country codes, tax-authority codes and portal URLs, VAT category codes, `tax_id_type` regexes, business-unit codes, approval-role codes, payment methods, and the five AI-session phase codes. Any new enum value must be added to `enums.yaml` *before* being used in a schema or record; `validate.py` enforces this.

2.8 Provenance

Each record stores the SHA-256 of its source document in `source_document.sha256`. Reproducing the corpus from the originals is therefore a single command:

```
find docs/arabic/source -type f -print0 \
  | xargs -0 shasum -a 256 \
  | diff - <(jq -r '.items[] | select(.sha256) | "\(.sha256)  \(.source_path)"' \
              docs/arabic/structured/corpus.json | sed 's|docs/arabic/source/||')
```

A clean diff proves the corpus is unaltered since the last specification snapshot.

2.9 Unified Schema Registry

As of 18 April 2026, all schemas in this specification are published in the unified schema registry at `docs/schema/`. Migration status is *complete* (see `docs/schema/registry.json` field `migration.status`). This consolidation addresses cross-document schema divergence identified in the specification review.

Registry index. `docs/schema/registry.json` — Master index of all schemas across domains.

Common types. `docs/schema/common/base-types.schema.json` — Shared definitions (`Metadata`, `Currency`, `TaxIdentifier`, `EntityParty`) referenced via JSON Schema `$ref`.

Consolidated enums. `docs/schema/common/enums.yaml` — Single source of truth for enumeration values including `countries`, `vat_category`, `supplier_type`, and `vat_thresholds` used by this specification.

Arabic schemas. `docs/schema/arabic/` — Domain-specific schemas: `invoice.schema.json`, `vat-checklist.schema.json`, `vendor.schema.json`, and `ocr-result.schema.json`. All marked `status: stable`.

The per-country VAT thresholds (e.g. BHD 500 for Bahrain SI/FI boundary) are defined in `docs/schema/common/enums.yaml\#/vat\_thresholds` rather than in domain-specific `enums.yaml` files. Validation:

```
python3 docs/schema/validate.py --registry-check
python3 docs/schema/validate.py --domain arabic \
    --schema invoice.schema.json --file invoice.json
```

CI runs `--registry-check` on every pull request touching `docs/schema/` or any dependent pipeline module.

Chapter 3

Arabic invoice AI pipeline

3.1 Pipeline at a glance

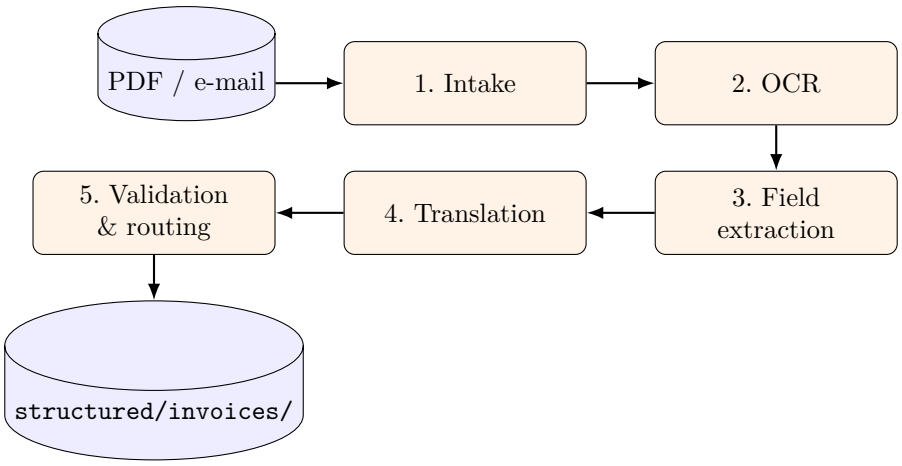

Figure 3.1: The five pipeline stages; each emits a well-typed payload to the next. Only Stage 5 writes to the canonical corpus.

3.2 Stage 1 – Intake

Inputs. A source file, either:

- a PDF dropped into a watched folder, or
- an e-mail attachment (e.g. the thread captured in `docs/arabic/source/FW- AI _ Arabic Invoice….eml`).

Outputs. A new entry staged for Stage 2 and a provisional `source_document` object:

```
{ "path": "docs/arabic/source/<file>", "sha256": "<sha>" }
```

The SHA-256 is computed first and compared against `docs/arabic/structured/corpus.json`; a match short-circuits re-processing (idempotence) and is the duplicate-payment control

(§6).

Precondition. The MIME detector must classify the file as one of `application/pdf`, the OOXML Word type (`application/vnd.openxmlformats-officedocument.wordprocessingml.document`), `text/plain`, or `message/rfc822`. All other types are rejected with reason UNSUPPORTED_MEDIA.

3.3 Stage 2 – OCR

Command. The corpus has been produced with:

```
pdftoppm -r 300 <input.pdf> work/page -png
for img in work/page-*.png; do
  tesseract "$img" stdout -l ara+eng >> <output>.txt
done
```

The combined `ara+eng` language model is mandatory: invoices routinely contain English brand names, Latin-script SKU codes, and numerals alongside Arabic prose.

Quality gate. Stage 2 emits a per-page confidence metric (mean of tesseract's `conf` values over non-empty tokens). Pages below **55%** confidence are flagged for human review; below **30%** the file is rejected with reason OCR_LOW_CONFIDENCE and routed to manual processing.

Calibration. The 30 %/55 % thresholds are preliminary estimates pending production pilot validation.

v1 status: These values were derived from three golden-set documents, which is insufficient for statistical confidence. The 30 % threshold represents the lowest observed confidence on pages that still yielded correctable records; 55 % is the lowest value achieving error-free extraction. **Both thresholds are TBD pending pilot:** production calibration requires ≥50 documents per country before these values can be considered validated. The pilot will also establish per-jurisdiction calibration as OCR quality varies with document source (e.g., ZATCA assessments vs. commercial invoices).

Artefact. The raw OCR text is written to `docs/arabic/extracted-text/<slug>.txt` and its path is recorded in `source_document.ocr_text_path`. This preserves the training signal for future model refinement.

3.4 Stage 3 – Schema-guided field extraction

Objective. Produce a candidate record conforming to `invoice.schema.json` from the OCR text. The extraction is *schema-guided*: the LLM prompt is constructed at run time by

serialising the schema (with `description` fields) so the model is told exactly which JSON keys it may emit. This is the single largest lever against hallucinated fields.

Prompt template. Pseudocode; the concrete implementation uses the Anthropic Messages API with structured-output tooling.

Listing 3.1: Schema-guided extraction prompt template.

```
SYSTEM: You extract tax-invoice fields into a JSON object that MUST
        validate against the provided JSON Schema. If a field is not
        present in the source, omit it -- NEVER invent a value.
USER:   <schema>
        ---
        OCR TEXT (Arabic may appear in RTL order; preserve spelling):
        <ocr_text>
        ---
        Emit JSON only. Set "metadata.language" to "ar", "en", or "ar+en"
        based on the dominant script.
```

Known-hard fields. The following fields are the most error-prone in the shipped corpus and must be handled with care:

Table 3.1: Extraction pitfalls observed in the corpus and mitigations.

Field	Pitfall	Mitigation
`electronic_id`	Tesseract maps the digit "1" to "I" in mixed alnum strings (e.g. `B1STE8`→`BISTE8`).	Validate against the regex `^[0-9A-Z]{26}$`; on mismatch, re-OCR at 600 dpi with `--psm 6`.
`totals.gross`	Arabic digits (٠١٢٣٤٥٦٧٨٩) may slip through unmapped.	Pre-tokenise: substitute Arabic-Indic digits with ASCII before parsing.
`vat_rate_percent`	Zero-rated vs exempt both show 0.	Disambiguate with `vat_category`: ZR vs EX drawn from supplier's activity code lookup.
`signature.kind`	ZATCA assessments have no signature block.	Default to SYSTEM_GENERATED for `document_kind` = VAT_ASSESSMENT.
`seller.name_en`	Pure-Arabic invoices lack an English name.	Leave empty; Stage 4 will populate via translation.

Output. A candidate record is written in memory only (never persisted yet) and passed to Stage 4.

3.5 Stage 4 – Arabic→English translation

Scope. This stage does *not* translate the whole document. It translates only the free-text fields that downstream AP actually uses in English:

- `seller.name_en` and `buyer.name_en` if absent;
- every `line_items[].description_en` if absent;

- `seller.address_en` and `buyer.address_en`.

Tax IDs, amounts, dates, and enum values are *never* passed through translation; they are copied verbatim.

Glossary. A fixed glossary enforces terminology consistency. Worked examples:

Table 3.2: Locked glossary. The right-hand column is what the translation agent *must* output.

Arabic	English (locked)
هيئة الزكاة والضريبة والجمارك	Zakat, Tax and Customs Authority (ZATCA)
بنك ستاندرد تشارترد	Standard Chartered Bank
ضريبة القيمة المضافة	Value-Added Tax (VAT)
اشعار صدور فاتورة	Invoice Issuance Notification
فاتورة ضريبية	Tax Invoice

Quality gate. Back-translation is run as a sanity check: the English output is translated back to Arabic and compared to the source Arabic string. The metric is the *normalised Levenshtein distance* $d = \mathrm{lev}(a, b)/\max(|a|, |b|)$ computed at the Unicode-codepoint level after Arabic NFKC normalisation (stripping tatweel and harakat). The 0.30 ceiling was chosen because the three golden-set Arabic strings have pairwise $d \leq 0.18$ between their recorded Arabic and a human-verified back-translation, leaving a $\geq 1.5\times$ safety margin. Distances above **0.30** are flagged (TRANSLATION_DRIFT) and the original Arabic is retained as authoritative.

3.6 Stage 5 – Validation and routing

Validation. The candidate record is validated with `validate.py` (Chapter 2). Any schema failure halts the pipeline and the record is written to a side-queue (`structured/invoices/rejected/`) with a `notes[]` entry summarising the failure.

Routing. On success, the pipeline consults the structured `bh-accounts-payable-doi.json` and the `workflows/` set to decide which state machine to instantiate:

1. If the counterparty is a tax authority (`supplier_type` = GOVERNMENT or `tax_authority` = ZATCA) ⇒ `vat-payment` workflow.
2. Else if there is a PO reference in `notes[]` ⇒ `po-eproc` workflow.
3. Else if a prior `manual-recurring` record exists for the same `vendor_id` ⇒ `manual-recurring` workflow.
4. Otherwise ⇒ `manual-one-off` workflow.

Commit. The record is written to `docs/arabic/structured/invoices/<slug>.json` and a new item is appended to `corpus.json`. This is the only stage in the pipeline that mutates the corpus.

3.7 Pipeline control plane

Figure 3.2 models the pipeline itself as a small state machine; each invoice occupies exactly one of these states at any moment. The control plane exposes three operations: `ingest(file)`, `retry(id)`, `release(id)`.

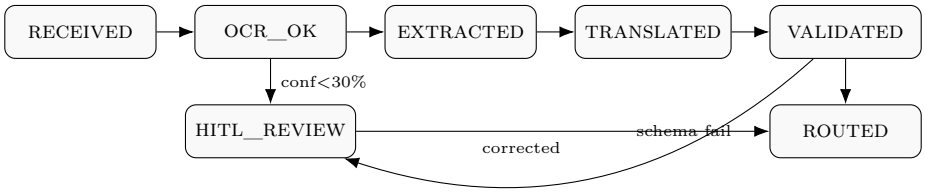

Figure 3.2: Pipeline control-plane states for a single invoice.

3.8 Evaluation harness

Golden set. The three Arabic records in `docs/arabic/structured/invoices/` form the regression golden set. Every code change must re-produce byte-identical records from the source PDFs; `validate.py` plus a `jq`-driven equality check is the verifier.

Per-field metrics. For each round of Session-5 testing the harness reports:

- **Per-field exact-match accuracy** (over the 27 scalar fields in `invoice.schema.json`).
- **Per-field F1** on `line_items[]` treated as sets (descriptions matched after glossary-aware fuzzing).
- **Human-approval rate.** Fraction of records released by HITL with zero edits. Target ≥ 0.75 after Session 4, ≥ 0.90 after Session 5.

Session acceptance. The five-session model from Table 1.2 is scheduled per country; the `ai_session_phases` enum in `enums.yaml` is the system of record. An AI rollout is complete for a country once the Session-5 pass meets the human-approval-rate target against a cohort of ≥ 5 stakeholders.

Chapter 4

Multi-country VAT compliance engine

4.1 Engine responsibilities

Given a validated invoice record, the VAT engine must:

1. select the correct regulator and ruleset from the record's `metadata.country` and `metadata.tax_authority`;
2. run every applicable rule, producing a `vat-checklist.schema.json` record with one item per rule;
3. decide a final `tax_type_classification` (VAT category code from `enums.yaml`);
4. compute any adjustments (withholding, reverse-charge accrual);
5. refuse to release the record for payment unless every mandatory rule emits Y or NA.

Rules are declarative data, not code: the engine is a rule interpreter. This isolates regulation change from engineering change.

4.2 Rule shape

A rule is a tuple (c, a, p, q, r) where:

- c — country (SA, EG, BH, QA);
- a — tax authority (ZATCA, ETA, NBR, QTA);
- p — a predicate over the invoice record (expressed as a JSON-Logic expression or a named helper);
- q — the checklist *question* emitted when the rule fires (human-readable string);
- r — a `result` $\in$ {Y, N, NA} computed by evaluating p and optionally a secondary predicate.

The engine iterates over the ruleset for (c, a) in a deterministic order and appends each

evaluated rule as an item in the output `checklist[]`. The final `tax_type_classification` is chosen by a *decision function* that maps the full result vector to a single VAT category.

4.3 Saudi Arabia (ZATCA)

4.3.1 Rules

The eleven-point ruleset is lifted verbatim from the Q3 2025 checklist and is shown in Table 4.1. Each row is one rule; the mapping from rule to `result` is given in the predicate column.

Table 4.1: KSA VAT rules derived from the Q3 2025 OTP checklist.

#	Question	Predicate / Source
1	Invoice in Arabic language	`metadata.language` $\in$ {AR, AR+EN}.
2	Addressed to Standard Chartered Capital	`buyer.tax_id` equals the entity's ZATCA TIN.
3	Date of issue present	`metadata.issue_date` set and well-formed.
4	Sequential number present	`metadata.electronic_id` or `metadata.invoice_number` set.
5	15-digit TIN on supplier side	`seller.tax_id` matches `^[0-9]{15}$`; if supplier is GOVERNMENT, emit NA.
6	Nature of goods/services present	At least one `line_items[].description*` non-empty.
7	VAT rate applied	`totals.vat_rate_percent` set and $\in \{0, 5, 15\}$.
8	Net amount	`totals.net` set.
9	Gross amount	`totals.gross` set.
10	VAT amount	`totals.vat_amount` set and equals round(`net` $\cdot\, r/100$).
11	Withholding tax	`withholding_tax.applicable`; NA if supplier is KSA-resident.

4.3.2 Decision function

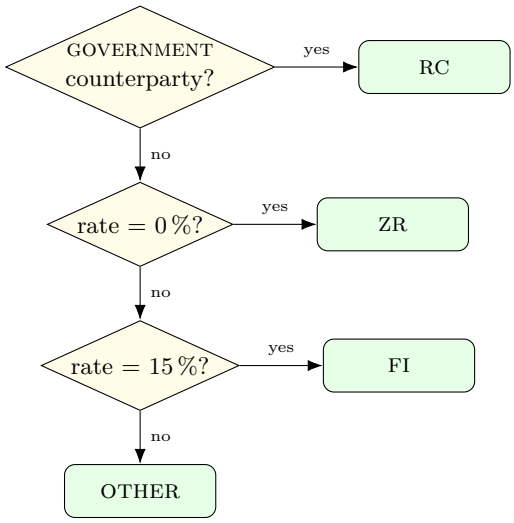

Figure 4.1: KSA decision function. The first matching branch determines the emitted `tax_type_classification`.

Listing 4.1: KSA decision logic applied to a validated invoice record.

```
if supplier.supplier_type == GOVERNMENT
     and tax_authority == ZATCA:
    classification = RC         # reverse-charge / government payment
elif totals.vat_rate_percent == 0
     and supplier.supplier_type != GOVERNMENT:
    classification = ZR
elif totals.vat_rate_percent == 15:
    classification = FI
elif totals.vat_rate_percent in {5, 10, 7, 14, 20}:
    # A rate that is legal in another jurisdiction but not in KSA.
    # Never silently classify: emit HOLD + human review.
    classification = OTHER
    verdict       = HOLD        # consumed by the AP engine
    reason        = "unexpected_vat_rate_for_ksa"
else:
    # Unknown rate (e.g. 3%, negative, null): always HOLD.
    classification = OTHER
    verdict       = HOLD
    reason        = "unknown_vat_rate"
```

4.3.3 Worked example

The Saudi Q3 2025 assessment (اشعار صدور فاتورة) has `supplier.supplier_type` = GOVERNMENT and `tax_authority` = ZATCA; the decision function correctly yields `tax_type_classification` = RC. All eleven rules fire; rules 5 and 11 are NA because the counterparty is ZATCA itself and the buyer is KSA-resident. The total `gross` = SAR 1,558,629.96 (due 2025-10-31) is posted as a reverse-charge VAT payment through the `vat-payment` workflow.

4.4 Egypt (ETA)

4.4.1 Rules

Table 4.2: ETA e-invoice rules.

#	Question	Predicate / Source
1	`electronic_id` well-formed	Matches `^[0-9A-Z]{26}$`.
2	Seller TRN well-formed	`seller.tax_id` matches `^[0-9]{9}$`.
3	Buyer TRN well-formed	`buyer.tax_id` matches `^[0-9]{9}$`.
4	Activity code present	`seller.activity_code` set.
5	Submission within 24h of issue	`submission_date` − `issue_date` $\leq$ 24 h.
6	VAT rate consistent	14% (standard) or 0% (exempt by activity lookup).
7	Line-item subtotals	Sum of `line_total` equals `totals.net`.

4.4.2 Decision function

```
if seller.activity_code in EXEMPT_ACTIVITIES:
```

```
    classification = EX
elif totals.vat_rate_percent == 14:
    classification = FI
elif totals.vat_rate_percent == 0:
    classification = ZR
else:
    classification = OTHER
```

The set `EXEMPT_ACTIVITIES` is a configured lookup. Activity `8890` ("Professional services", as used by the Chapter Zero corporate-membership invoice) is included.

4.4.3 Worked examples

Seal invoice (`3CPJR5CQ…K10`). All seven rules pass; activity code `1811` is not exempt; `totals.vat_rate_percent = 14`; the decision function yields FI (Full Tax Invoice). Gross `EGP 661,200` posts to the PO e-Proc or manual path depending on the presence of a PO reference.

Corporate membership (`B1STE8RH…SJ10`). Activity code `8890` is in the exempt set; rule 6 still passes because 0% with an exempt code is consistent; decision yields EX. No VAT accrues and the routing is `manual-one-off`.

4.5 Bahrain (NBR)

4.5.1 Rules

Derived from Activity 3 of the Bahrain DOI (`docs/arabic/structured/policies/bh-accounts-payable-doi.json`). The NBR ruleset is wider than KSA's eleven points because the DOI distinguishes ten VAT invoice types.

Table 4.3: NBR VAT invoice types recognised in Bahrain AP. Each row maps to a `vat_category` enum value.

DOI name	Enum	Selection criterion
Full Tax Invoice	FI	VAT-registered supplier, $\geq$ BHD 500.
Simplified Tax Invoice	SI	Transaction **under BHD 500**.
Unregistered Vendor Purchases	UR	Supplier not VAT-registered.
Zero Rated Purchases	ZR	Transport, food, healthcare, education, construction, oil/gas.
Exempt Purchases	EX	Real estate, financial services.
Tax Credit Notes	TCN	Negative-amount adjustment to prior invoice.
Reverse Charge Invoices	RC	Overseas supplier; buyer accounts for VAT.
GCC Tax Invoice	FI	Intra-GCC supplier (handled as FI with a GCC flag).
Inter Business Unit Transfers	CBR	Routed via GBS APU CBR team.
Out of Scope	OS	Pre-Jan 2019 vouchers, MPV vouchers.
Other	OTHER	EWA utility bills.

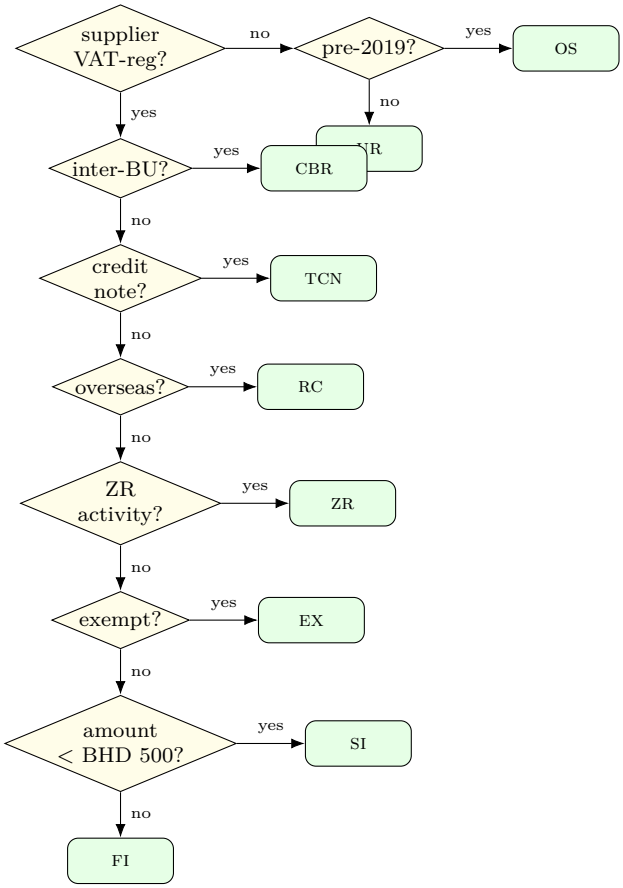

Figure 4.2: Bahrain NBR classification cascade. The first branch whose predicate holds emits the `vat_category` and short-circuits the remaining tests.

4.5.2 BHD 500 threshold

The DOI fixes BHD 500 as the SI/FI threshold for Bahrain. This value is stored in the per-country engine configuration file `vat-thresholds.yaml` (not in `enums.yaml`, which holds only enum *types*). The threshold structure is:

```yaml
thresholds:
  BH:
    si_fi_boundary: 500
    currency: BHD
```

Regulator updates are applied by editing this file; a CI check validates that all threshold values match the authoritative DOI documents.

4.6 Qatar (QTA)

Qatar does not currently impose VAT. The engine therefore runs a single rule — *payment is to a valid counterparty with a QCSD-registered account* — and classifies the record as OTHER unless a future VAT regime is detected (`metadata.tax_authority` ≠ QTA). The docs/arabic/`structured/payment-instructions/qa-qcsd-payment-instruction.json` record is the reference.

4.7 Withholding tax

Withholding rules are modelled as a secondary ruleset keyed on *supplier residence*. The engine emits a `withholding_tax` object with `applicable`, `rate_percent`, and `amount`. KSA-resident suppliers yield `applicable = false` (NA), mirroring the Q3 2025 checklist. Non-resident, non-GCC suppliers trigger a country-specific rate (e.g. 5% KSA royalty, 15% services) that must be explicitly configured in `enums.yaml` before the engine can emit Y/N.

4.8 Engine outputs

For every invoice the engine produces:

- a `vat-checklist` record stored at `structured/vat-checklists/<slug>.json`;
- an updated `invoice` record with a populated `totals.vat_category`;
- a verdict of RELEASE or HOLD consumed by the approval engine (Chapter 5).

A hold is produced whenever any rule resolves N. The engine must never up-grade a HOLD to RELEASE without a human recorded in the corresponding workflow transition.

Chapter 5

AP workflow and approval engine

5.1 Engine responsibilities

Given a validated invoice record and the VAT engine's verdict (Chapter 4), the approval engine must:

1. select the correct workflow from `docs/arabic/structured/workflows/`;
2. instantiate the state machine with the record;
3. drive transitions as approvals arrive (human or system);
4. at every transition, verify the `delegation_of_authority` is respected for the current LCY amount;
5. on reaching a terminal state, emit the payment instruction conforming to `payment-instruction.schema.json`.

The engine is a *pure* workflow executor; it never invents fields.

5.2 Activity view of the Bahrain DOI

Table 5.1 is the unabridged activity map from `docs/arabic/structured/policies/bh-accounts-payable-doi.json`; the engine maps each activity onto one or more workflow paths.

5.3 Workflow selection

5.4 `po-eproc` state machine

Preconditions.

- `tax_q_review` → `posted`: VAT category assigned and FCY revalued at central-bank rate on `issue_date`.
- `posted` → `paid`: LCY amount within DOA per GDAM.

Table 5.1: The seven DOI activities and the workflow paths they drive.

#	**Activity**	**Description (condensed)**	**Workflow**
1	Introduction	DOI purpose & Group alignment.	—
2	Group systems	Mandate eProc/T&E/SODA; anything else = manual.	—
3	PO invoices	eProc route; Tax-Q assigns POP/VAT; FCY at central-bank rate.	`po-eproc`
4	Manual invoices (non-eProc)	CC + supply chain + CFO approvals; BH_APUeInvoice mailbox.	`manual-one-off`
5	In-country manual payments	One-off: MD+CFO + 48h screening. Recurring: dispensation + eOPS.	`manual-one-off`, `manual-recurring`
6	CBR	Centrally managed by GBS APU CBR team.	(central, out of this engine)
7	Suspense clearing	Monthly PSGL↔eBBS reconciliation.	(batch, not per-invoice)

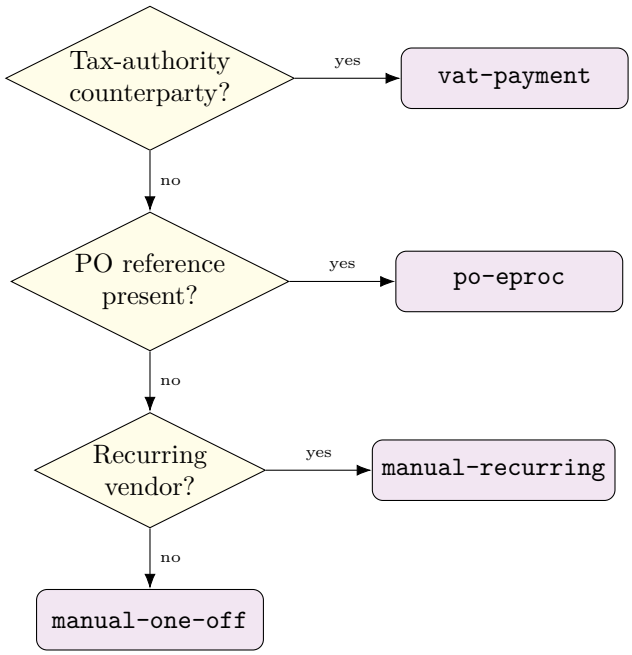

Figure 5.1: Workflow selection decision tree applied at Stage 5 of the AI pipeline (Chapter 3).

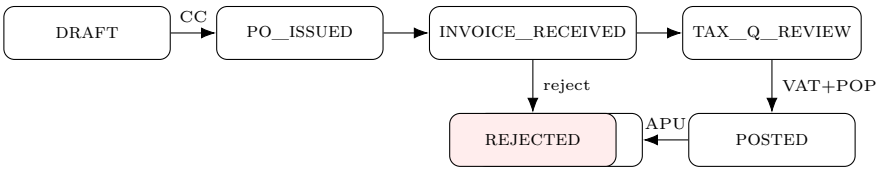

Figure 5.2: `po-eproc` state machine (`docs/arabic/structured/workflows/po-eproc.json`).

5.5 `manual-one-off` state machine

Figure 5.3 captures Activity 5's one-off branch: evidence of one-off nature, vendor screening refreshed within 48 hours, MD and CFO approvals, and account coding captured in the approval document.

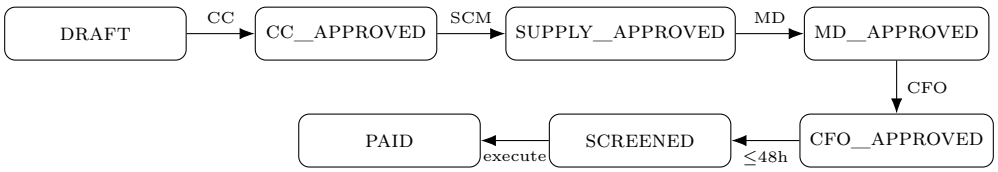

Figure 5.3: `manual-one-off` (docs/arabic/`structured/workflows/manual-one-off.json`).

5.6 `manual-recurring` state machine

Precondition. A written dispensation from the policy owner is *mandatory* before entry. This is enforced by the engine: the record cannot leave DRAFT until a `dispensation_document_path` is recorded in `notes[]`. Post-payment, the record must reach QUARTERLY_REVIEWED within the following quarter, triggered by the CFO's quarterly log retrieval.

5.7 `vat-payment` state machine

Used when the counterparty is a tax authority (`supplier.supplier_type` = GOVERNMENT). The KSA Q3 2025 assessment walks this workflow from ASSESSMENT_RECEIVED → CHECKLIST_VALIDATED (upon 11/11 rules resolved) → CFO_APPROVED → PAID → POSTED.

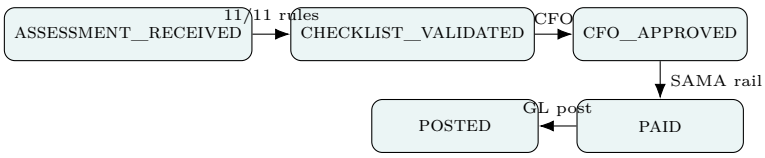

Figure 5.4: `vat-payment`: linear path walked by the ZATCA Q3 2025 assessment. Entry requires every VAT checklist rule to resolve to Y or NA.

5.8 Delegation-of-authority

The engine enforces DOA as a linear scan of the workflow's `delegation_of_authority` array, sorted ascending by `max_amount`. The role required is the first entry whose `max_amount` is ≥ the invoice's LCY value. An FX conversion (at the central-bank rate on `issue_date`) is applied before comparison.

Table 5.2 shows the shipped thresholds.

Table 5.2: Default DOA thresholds per workflow.

Workflow	Currency	Max LCY	Required role
po-eproc	BHD	10,000	CC_OWNER
po-eproc	BHD	100,000	SUPPLY_CHAIN_MGR
po-eproc	BHD	1,000,000	CFO
manual-one-off	BHD	500	SUPPLY_CHAIN_MGR
manual-one-off	BHD	50,000	CFO
manual-one-off	BHD	1,000,000	MD
manual-recurring	BHD	1,000,000	CFO
vat-payment	SAR	10,000,000	CFO

5.9 Exceptions

The shipped workflows carry an `exceptions[]` array. The most important entry is the GSAM legal/professional fees exception (`manual-one-off`): these are *not* covered by the default GDAM approval limits and must be routed to a GSAM-specific approver. The engine consults `exceptions[]` before applying DOA.

5.10 Payment-suspense reconciliation

Activity 7 is operated as a *batch* procedure; the approval engine does not execute it per invoice. Reconciliation compares the PSGL extract against the eBBS extract on the accounts listed in the DOI:

- DBU BHD suspense: `09-0906743-50`
- DBU USD suspense: `09-0906743-50`

The aging escalation grid (30/60/90 day buckets) drives automated e-mails to the DBU finance manager; items aged >90 days escalate to the CFO.

5.11 Auditability

Every transition records a structured event: `(workflow_id, from, to, actor_psid, timestamp, evidence_refs)`. The concatenation of events is a tamper-evident log because each event hashes its predecessor. This is what downstream auditors consume; it is *not* part of the record schema but sits alongside the `structured/` corpus in an append-only `events/` stream. The event contract specified here is in scope (Chapter 6, "Audit event log"); the storage substrate, retention policy, and restoration runbook are owned by GBS APU Audit and documented in *SCB-APU-AUDIT-001*.

Chapter 6

Controls, testing, and acceptance

6.1 Control catalogue

The Bahrain DOI's seven controls (`docs/arabic/structured/policies/bh-accounts-payable-doi.json`, `controls[]`) are reproduced in Table 6.1 with the engine mechanism that enforces each.

Table 6.1: Control catalogue and enforcement mechanisms.

ID	Activity	Description	Enforced by
C1	4	All required approvals present before payment release.	Workflow engine cannot transition to PAID until every required-role approval has fired.
C2	5	Vendor screening revalidated within 48 hours of payment.	Precondition on `cfo_approved`→SCREENED transition (`manual-one-off`).
C3	4	Amount payable within DOA per GDAM.	Linear DOA scan before any approval transition.
C4	5	Dispensation in place for recurring manual payments.	Precondition on DRAFT→DISPENSATION_OK in `manual-recurring`.
C5	7	Monthly PSGL↔eBBS reconciliation cleared within aging thresholds.	Batch job; escalation grid e-mails DBU FM / CFO.
C6	3	FCY invoices revalued at central-bank rate on invoice date.	VAT engine runs FCY→LCY *before* DOA check.
C7	4	Duplicate-payment check.	Intake Stage 1 short-circuits on matching `source_document.sha256`.

6.2 Testing strategy

Testing follows the five-session engagement model (Table 1.2). For each country rollout:

Session 1 — Understand. Collect ≥5 sample invoices (regulator-issued if available), normalise into the corpus, confirm per-country ruleset is complete.

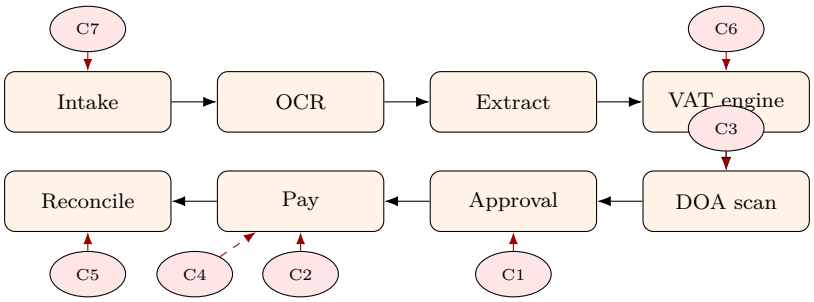

Figure 6.1: Control enforcement points along the end-to-end invoice lifecycle. Dashed arrows show where each DOI control (Table 6.1) is evaluated.

Session 2 — Sketch. Walk the end-to-end pipeline on paper using one sample invoice. Identify OCR pitfalls (Table 3.1) specific to the country.

Session 3 — Decide. Freeze the rule precedence and the decision function for the VAT engine (§4).

Session 4 — Prototype. Run the full pipeline on the cohort; review each extracted record with the local finance team. Human-approval rate target ≥ 0.75.

Session 5 — Testing. Independent run with ≥ 5 stakeholders; target ≥ 0.90 human-approval with zero-edit rate measured.

6.3 Regression golden set

The three Arabic records already in `docs/arabic/structured/invoices/` (Egypt ETA $\times$ 2, Saudi ZATCA $\times$ 1) together with the KSA checklist and the two Qatar payment instructions constitute the regression golden set.

Invariants. Every change to the pipeline must preserve:

1. byte-identical `source_document.sha256` for every corpus record (source files are immutable);
2. 100 % schema validity under `validate.py`;
3. identical `tax_type_classification` for every invoice record;
4. identical selected workflow for every record.

Runner. `validate.py` plus the following `jq` equality check is the regression harness:

```
# Per-field equality of every golden-set record between a reference
# snapshot and the pipeline output.
diff \
  <(jq -S . docs/arabic/structured/invoices/*.json) \
  <(for p in docs/arabic/source/*.pdf; do
      ./pipeline extract "$p"
    done | jq -S .)
```

A non-empty diff is a regression.

6.4 Acceptance criteria per workstream

AI pipeline (Chapter 3).

- Corpus integrity check passes (every `corpus.json` path exists; every SHA-256 matches).
- For every source PDF: OCR confidence $\geq 30\%$ on every page, schema-valid record emitted.
- Translation back-translation distance ≤ 0.30 on every non-empty `description_en`.

VAT engine (Chapter 4).

- KSA: all 11 checklist rules evaluated; decision function emits expected VAT category for every golden invoice.
- Egypt: `EXEMPT_ACTIVITIES` lookup table complete for activities present in the corpus.
- Bahrain: each of the 11 NBR invoice types has a corresponding enum code.

AP engine (Chapter 5).

- Every workflow record loadable and schema-valid.
- DOA scan deterministic under round-trip FX conversion.
- Workflow-selection decision tree (Figure 5.1) yields the documented workflow for every corpus record.

6.5 Traceability matrix

Table 6.2 links each specification section to the source document that justifies it and to the structured record that realises it. Every row must be present for a full audit pass.

6.6 Operational readiness

A country is declared *production-ready* once:

1. the regression golden set runs clean for 30 consecutive days in the UAT environment;
2. Session 5 acceptance meets the human-approval-rate target (Chapter 3);
3. every VAT rule for the country is present in the ruleset and has at least one passing test case;
4. the country CFO signs off on the workflow configuration, recorded as a `CFO` role entry on the `vat-payment` or `manual-one-off` workflow golden record.

Table 6.2: Traceability of specification to source document and structured record.

Section	Requirement	Source	Structured record
Chapter 1	Countries in scope	Manoj e-mail, 2025-11-23	`enums.yaml` (countries)
Chapter 2	Canonical invoice fields	EG ETA PDFs, ZATCA PDF	`schema/invoice.schema.json`
Chapter 2	11-point checklist shape	Saudi OTP/VAT Q3 2025	`schema/vat-checklist.schema.json`
Chapter 3	OCR command line	— (new)	`extracted-text/*.txt`
Chapter 3	Translation glossary	اشعار صدور فاتورة (005).pdf	`invoices/sa-zatca-ishaar-005.json`
Chapter 4	KSA rules (1–11)	Saudi OTP/VAT Q3 2025	`vat-checklists/sa-otp-vat-q3-2025.json`
Chapter 4	ETA 26-char ID regex	EG ETA PDFs	`enums.yaml` (tax_authorities.ETA)
Chapter 4	NBR 11 invoice types	AP DOI Activity 3	`policies/bh-accounts-payable-doi.json`
Chapter 5	Activities 1–7	AP DOI	`policies/bh-accounts-payable-doi.json`
Chapter 5	`po-eproc` state machine	AP DOI Activity 3	`workflows/po-eproc.json`
Chapter 5	`manual-one-off` state machine	AP DOI Activity 5 (one-off)	`workflows/manual-one-off.json`
Chapter 5	`manual-recurring` state machine	AP DOI Activity 5 (recurring)	`workflows/manual-recurring.json`
Chapter 5	`vat-payment` state machine	KSA Q3 2025 flow + ZATCA assessment	`workflows/vat-payment.json`
Chapter 6	Controls C1–C7	AP DOI + KSA checklist + observations	`policies/bh-accounts-payable-doi.json`

6.7 Deployment and rollback

Environments. Three are in scope:

dev. Developer workstations and CI; reads the corpus from the Git working tree. No live invoices.

uat. Shared test environment reading a frozen copy of the corpus; live DOI activity stops at CFO_APPROVED (no SAMA/SWIFT rail). Country finance and GBS APU run Session 4/5 here.

prod. Live environment; the only one authorised to transition to PAID.

Sign-off matrix. Each promotion requires a two-party approval recorded as an event in the `events/` stream (Chapter 5):

Transition	Approvers	Evidence
DEV→UAT	Tech lead + QA lead	Green CI build, clean regression.
UAT→PROD	Country CFO + GBS APU head	Session 5 acceptance report; 30-day UAT stability; CFO workflow golden record.

Rollback. Deployments are blue/green. The engine, rules, and schemas are versioned with a content-addressed tag (SHA-256 of `structured/corpus.json` at release time). Rollback is a one-command revert to the previous tag; no database migration is required because the

canonical corpus is append-only. Any in-flight invoice whose workflow record references a schema version that was rolled back is automatically held in AWAITING_REVIEW.

Change control. Ruleset edits (Chapter 4) and workflow edits (Chapter 5) are routed through the same two-party sign-off; schema edits require an additional sign-off from the AI pipeline owner because they invalidate the regression golden set (§3).

6.8 Audit event log

The approval engine emits an append-only event per transition (`workflow_id`, `from`, `to`, `actor_psid`, `timestamp`, `evidence_refs`), hashing its predecessor so the stream is tamper-evident. The stream is *in scope* for compliance purposes but the storage substrate (append-only object store, retention policy, restoration runbook) is owned by the GBS APU audit team and specified in *SCB-APU-AUDIT-001*, referenced but not reproduced here. The engine guarantees:

- every workflow transition in Chapter 5 emits exactly one event;
- events are linearly hash-chained;
- the event stream is the sole source of truth for DOA compliance (controls C1–C4 in Table 6.1).

Chapter 7

References

This appendix lists every file that backs a normative statement in the specification. Paths are relative to the repository root. SHA-256 hashes are taken from docs/arabic/structured/corpus.json and are the ground truth for the provenance check of Chapter 2.

7.1 Source documents

Short name	Path	SHA-256 (prefix)
EG ETA seal invoice	docs/arabic/source/3CPJR5CQ4PNNN9A4XHBB370K10.pdf	2b912687fa94…
EG ETA corp membership	docs/arabic/source/B1STE8RHNWZY9TYEXDS6WSZJ10.pdf	18eb35671131…
SA ZATCA assessment	docs/arabic/source/ اشعار صدور فاتورة (005).pdf	cbbb3f702ad5…
QA EDAA annual fee	docs/arabic/source/EDAAAnnualFeeInvoice2025.pdf	cf91c2a43574…
QA QCSD payment instr.	docs/arabic/source/Instructionforpaymenttovendor(QCSD).pdf	39ed62690d0f…
SA OTP VAT Q3 2025	docs/arabic/source/Saudi-OTPEntryprocessing-VATQ3'25(1).docx	3a6c8f37528f…
BH AP DOI	docs/arabic/source/AccountsPayableDOI.docx	128ff6792798…
AI initiative e-mail	docs/arabic/source/FW-AI_ArabicInvoice...eml	e01ba78d5ca5…

7.2 Derived text

Every Arabic PDF has a UTF-8 OCR artefact in docs/arabic/extracted-text/, produced with `pdftoppm -r 300 | tesseract -l ara+eng`. English PDFs use `pdftotext -layout`; Word documents use `pandoc -t plain`; the e-mail is decomposed by a small Python script driven by the standard-library `email` module.

7.3 Structured records

All of the following are JSON records validated by docs/arabic/structured/validate.py.

Type	Path
Invoice	structured/invoices/eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10.json
Invoice	structured/invoices/eg-eta-B1STE8RHNWZY9TYEXDS6WSZJ10.json
Invoice (assessment)	structured/invoices/sa-zatca-ishaar-005.json
Payment instruction	structured/payment-instructions/qa-edaa-annual-fee-2025.json
Payment instruction	structured/payment-instructions/qa-qcsd-payment-instruction.json
VAT checklist	structured/vat-checklists/sa-otp-vat-q3-2025.json
DOI policy	structured/policies/bh-accounts-payable-doi.json
Workflow	structured/workflows/po-eproc.json
Workflow	structured/workflows/manual-one-off.json
Workflow	structured/workflows/manual-recurring.json
Workflow	structured/workflows/vat-payment.json
Index	structured/corpus.json
Enumerations	structured/enums.yaml

7.4 Tools

Tool	Purpose
pdftoppm	Rasterise PDF pages to 300 dpi PNG for OCR.
tesseract	Bilingual OCR with -l ara+eng.
pdftotext	Direct text extraction for born-digital English PDFs.
pandoc	Convert .docx to plain text.
Python email	Parse the .eml envelope and MIME parts.
jq	Query and cross-check corpus.json.
jsonschema / referencing (Python)	Offline schema validation with a local registry.
xelatex / tectonic	Build this document (polyglossia, bidi support).

7.5 Minimal corpus excerpt

The entire corpus is in-tree under docs/arabic/; for readers without access to the Git repository, Listing 7.1 reproduces the first three items of docs/arabic/structured/corpus.json verbatim. This is a sufficient starting point to reconstruct the shape of the index; the full file contains 12 items covering every source in Table 2.1.

Listing 7.1: First three items of `corpus.json`; identical SHA-256 values are reproduced in Chapter 7.

```
{
  "schema_version": "1.0",
  "items": [
    {
      "id": "eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10",
      "type": "invoice",
      "country": "EG",
      "tax_authority": "ETA",
      "sha256": "2b912687fa94...",
      "source_path":     "docs/arabic/source/3CPJR5CQ...K10.pdf",
      "text_path":       "docs/arabic/extracted-text/eg-eta-...K10.txt",
      "structured_path": "docs/arabic/structured/invoices/eg-eta-...K10.json"
    },
    {
      "id": "sa-zatca-ishaar-005",
      "type": "invoice",
      "country": "SA",
      "tax_authority": "ZATCA",
      "sha256": "cbbb3f702ad5...",
      "source_path":     "docs/arabic/source/<arabic-name>.pdf",
      "text_path":       "docs/arabic/extracted-text/sa-zatca-ishaar-005.txt",
      "structured_path": "docs/arabic/structured/invoices/sa-zatca-ishaar-005.json"
    },
    {
      "id": "qa-edaa-annual-fee-2025",
      "type": "payment_instruction",
      "country": "QA",
      "tax_authority": null,
      "sha256": "cf91c2a43574..."
    }
  ]
}
```

7.6 Out-of-scope assets

The following files sit in docs/arabic/source/ but are *not* part of the corpus and are not referenced by any record:

- `Business Instruction Template - V1.0 (002) (1).xlsm` — encrypted CDFV2 workbook; cannot be opened without the owner's password.
- `EDAA Annual Fee Invoice 2025[65].pdf` — byte-identical duplicate of `EDAA Annual Fee Invoice 2025.pdf`.

Chapter 8

VAT Module Implementation Specification

This chapter provides the technical implementation specification for the VAT processing module, addressing the review finding that "VAT engine rule format is underspecified." The specification enables developers to build conformant implementations.

8.1 Module architecture

The VAT module comprises four components:

1. **RulesEngine** — evaluates JSON-Logic expressions against invoice data and produces checklist items;
2. **RegistrationValidator** — validates VAT registration numbers against country-specific formats and (optionally) authority APIs;
3. **Calculator** — computes expected VAT amounts given rates and subtotals;
4. **ActivityLookup** — maps business activity codes to VAT treatment categories.

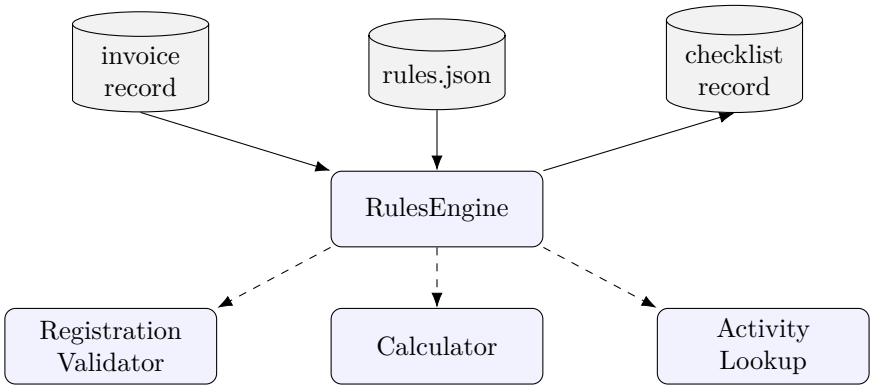

Figure 8.1: VAT module component diagram. Dashed arrows indicate helper invocations from the rules engine.

8.2 API contracts

Each helper function has a defined API contract. Implementations **must** conform to these signatures to be invoked by the rules engine.

8.2.1 Registration validation

Listing 8.1: Registration validator API contract.

```
def check_vat_registration_validity(
    tax_id: str,
    country: str  # ISO 3166-1 alpha-2
) -> ValidationResult:
    """
    Validate a VAT registration number.

    Returns:
        ValidationResult with fields:
        - valid: bool
        - format_valid: bool
        - authority_confirmed: Optional[bool]
        - error_code: Optional[str]
    """
```

Table 8.1: VAT registration number formats by country.

Country	Regex	Notes
SA	`^[0-9]{15}$`	15-digit ZATCA TIN
EG	`^[0-9]{9}$`	9-digit ETA TRN
BH	`^[0-9]{9,15}$`	NBR VAT number (variable length)
QA	`^[A-Z0-9]{10}$`	QCSD registration (non-VAT)

8.2.2 VAT calculation

Listing 8.2: VAT calculator API contract.

```
def calculate_vat_amount(
    subtotal: Decimal,
    vat_rate_percent: Decimal
) -> CalculationResult:
    """
    Calculate expected VAT amount.

    Returns:
        CalculationResult with fields:
        - vat_amount: Decimal (rounded to 2 decimal places)
        - gross_amount: Decimal
        - calculation_method: str  # "standard" | "reverse"
    """
```

The calculator **must** use banker's rounding (ROUND_HALF_EVEN) and **must** handle the following edge cases:

1. Negative subtotals (credit notes): return negative VAT amount
2. Zero rate: return VAT amount of exactly 0.00
3. Rate > 100%: reject with error code `INVALID_RATE`

8.2.3 Activity lookup

Listing 8.3: Activity lookup API contract.

```
def lookup_activity_code_treatment(
    activity_code: str,
    country: str  # ISO 3166-1 alpha-2
) -> TreatmentResult:
    """
    Determine VAT treatment from business activity code.

    Returns:
        TreatmentResult with fields:
        - vat_category: str  # ZR, EX, FI, etc.
        - rate_percent: Optional[Decimal]
        - source: str  # regulation reference
    """
```

Table 8.2: Activity code to VAT treatment mapping (Egypt, partial).

Code	Description	Treatment	Source
1811	Printing services	FI (14%)	ETA Circular 2019/1
8890	Professional services	EX (exempt)	ETA Circular 2020/3
4911	Passenger rail transport	ZR (0%)	VAT Law Art. 23
6411	Banking services	EX (exempt)	VAT Law Art. 29

8.3 JSON-Logic expression specification

The rules engine evaluates predicates expressed in JSON-Logic format (`https://jsonlogic.com`). This section specifies the supported operators and variable paths.

8.3.1 Supported operators

8.3.2 Variable paths

Variable paths use dot notation to access nested fields in the invoice record. The root object is the validated invoice conforming to `invoice.schema.json`.

Listing 8.4: Example variable paths for invoice data.

```
totals.gross           # Invoice gross amount
totals.vat_rate_percent  # VAT rate as percentage
```

Table 8.3: JSON-Logic operators supported by the rules engine.

Category	Operators	Notes
Comparison	`==`, `===`, `!=`, `!==`	Strict equality recommended
	`<`, `<=`, `>`, `>=`	Numeric comparison
Logic	`and`, `or`, `!`, `!!`	Boolean operations
	`if`	Ternary conditional
String	`in`, `cat`, `substr`	String operations
Array	`in`, `merge`, `map`, `filter`	Array operations
Arithmetic	`+`, `-`, `*`, `/`, `%`	Basic arithmetic
Data	`var`, `missing`, `missing_some`	Variable access

```
seller.tax_id          # Seller VAT registration number
seller.country         # Seller country code
buyer.country          # Buyer country code
metadata.issue_date    # Invoice issue date
line_items.0.description  # First line item description
```

8.3.3 Named helper invocation

When a predicate requires complex logic beyond JSON-Logic capabilities, the engine invokes a named helper. The syntax is:

Listing 8.5: Named helper invocation in rule predicate.

```
{
  "predicate": {
    "type": "named_helper",
    "name": "check_vat_registration_validity",
    "args": {
      "tax_id": {"var": "seller.tax_id"},
      "country": {"var": "seller.country"}
    }
  }
}
```

8.4 Error handling specification

The module **must** handle errors gracefully without crashing the parent invoice processing pipeline.

8.4.1 Error codes

8.4.2 Fallback behaviour

When an error occurs:

1. Log the error with full context (invoice ID, field values, stack trace)

Table 8.4: VAT module error codes.

Code	Severity	Description
`VAT_E001`	ERROR	Invalid VAT registration format
`VAT_E002`	ERROR	Unknown country code
`VAT_E003`	ERROR	Invalid VAT rate (negative or >100%)
`VAT_E004`	ERROR	JSON-Logic parse failure
`VAT_W001`	WARNING	Authority API unavailable (fallback to format check)
`VAT_W002`	WARNING	Activity code not in lookup table
`VAT_I001`	INFO	Rate changed since last validation

2. Set the rule result to HOLD (never silently pass)
3. Populate `checklist[].error_code` with the appropriate code
4. Continue processing remaining rules (fail-open for diagnostics)
5. Return the partial checklist to the caller

8.5 Performance requirements

Table 8.5: VAT module performance requirements.

Metric	Target	Measurement
p95 latency per invoice	< 100 ms	Excluding authority API calls
p95 latency with API	< 2000 ms	Including authority validation
Memory per invoice	< 10 MB	Peak allocation during processing
Throughput	> 100/sec	Single-threaded, no API calls

8.6 Implementation checklist

Developers **must** verify the following before marking the module complete:

- [ ] All four helper functions implemented with correct signatures
- [ ] Unit tests achieve ≥90% line coverage
- [ ] All country-specific regex patterns validated against test data
- [ ] JSON-Logic parser handles all operators in Table 8.3
- [ ] Error codes match Table 8.4
- [ ] Performance meets targets in Table 8.5
- [ ] Integration test with real invoice records from each country
- [ ] Security review completed (input validation, injection prevention)

Chapter 9

Testing Specification

This chapter specifies the testing requirements for the Arabic AP invoice processing pipeline, including test cases, coverage targets, and CI/CD integration.

9.1 Test coverage requirements

Table 9.1: Arabic AP test coverage targets.

Component	Line %	Branch %	Notes
OCR extractor	85	75	
VAT rules engine	90	85	Critical path
Invoice validator	90	85	Schema + business rules
AP workflow engine	80	70	
Overall minimum	**85**	**75**	Enforced in CI

9.2 Test fixtures

Fixtures are located in `tests/fixtures/arabic/`.

9.2.1 Invoice fixtures

Table 9.2: Required invoice fixture categories.

Category	Count	Coverage
Standard invoices	20	5 per country (SA, EG, BH, QA)
Edge cases	15	Missing fields, unusual formats
Invalid invoices	10	Schema violations, invalid TINs
Credit notes	5	Negative amounts, adjustments
Multi-currency	5	Non-local currency invoices

9.2.2 OCR fixtures

OCR fixtures cover all confidence thresholds:

- High quality (>85%): 10 samples
- Medium quality (55–85%): 10 samples
- Low quality (30–55%): 5 samples
- Very low quality (<30%): 5 samples

9.3 Unit test cases

9.3.1 Invoice extraction

ID	Priority	Description
AP-EXT-001	P1	Extract all fields from standard Saudi invoice
AP-EXT-002	P1	Extract all fields from standard Egyptian invoice
AP-EXT-003	P1	Extract all fields from standard Bahrain invoice
AP-EXT-004	P1	Extract all fields from Qatar payment instruction
AP-EXT-005	P1	Handle missing optional fields gracefully
AP-EXT-006	P2	Handle multi-page invoice
AP-EXT-007	P2	Handle poor quality scan (<55% confidence)
AP-EXT-008	P2	Handle Arabic-only invoice
AP-EXT-009	P2	Handle Arabic+English bilingual invoice
AP-EXT-010	P3	Handle unusual date formats

9.3.2 VAT validation

ID	Priority	Description
AP-VAT-001	P1	Validate 15-digit Saudi TIN format
AP-VAT-002	P1	Validate 9-digit Egyptian TRN format
AP-VAT-003	P1	Calculate VAT amount matches invoice
AP-VAT-004	P1	Apply correct VAT rate by country
AP-VAT-005	P1	Identify reverse-charge transactions
AP-VAT-006	P2	Handle zero-rated supplies
AP-VAT-007	P2	Handle exempt supplies
AP-VAT-008	P2	Apply withholding tax rules
AP-VAT-009	P3	Handle credit notes correctly
AP-VAT-010	P3	Handle inter-GCC transactions

9.4 Integration test suite

```
pytest tests/integration/arabic/ -v --tb=short
```

Expected duration: 15 minutes.

9.4.1 End-to-end scenarios

1. **Happy path**: Invoice scan → OCR → extraction → VAT validation → approval workflow
2. **Manual review**: Low-confidence extraction triggers human review
3. **VAT hold**: Rule failure triggers hold and escalation
4. **Multi-country**: Invoices from all four countries processed correctly

9.5 Performance benchmarks

All p95 targets in Table 9.3 are measured on the *reference hardware profile* in Section 9.5.1. CI enforces each target by failing the performance regression job when the measured p95 exceeds the target by more than 20 % on three consecutive runs on identical hardware.

Table 9.3: Arabic AP performance targets on reference hardware.

Operation	p95 Target	Throughput	Notes
OCR extraction	3,000 ms	200/hr	Per page
Field extraction (LLM)	5,000 ms	100/hr	Per invoice
VAT rule evaluation	100 ms	1,000/sec	No external API
Full pipeline E2E	10,000 ms	50/hr	Including all steps

9.5.1 Reference hardware profile

GPU. 1× NVIDIA A100 40 GB SXM4 *or* L40S 48 GB. Shared with Simula and TB; reservation is not required for single-invoice latency targets.

OCR engine. Tesseract 5.3.x with Arabic language pack (`ara`, `ara_old`), CPU-only. A 2-page invoice at 300 dpi is the reference document size.

LLM. `Qwen2.5-14B-Instruct` served via vLLM 0.6.x, `max_model_len=8192`, 14-field extraction prompt (∼1,200 input tokens, ∼400 output tokens).

CPU / RAM. 8 vCPU, 32 GB RAM host.

Storage. NVMe SSD; invoice PDFs streamed from object storage with 2 Gbps link.

Network. Co-located vLLM; RTT <0.5 ms. For remote LLM endpoints, add measured RTT to the field-extraction p95.

Running on consumer-grade GPUs (e.g. T4 15 GB) is not supported for the 14B model; a fallback profile using `Qwen2.5-7B-Instruct` is documented in `docs/arabic/runbooks/perf-fallback.md` with correspondingly relaxed p95 targets (field extraction ≤8,000 ms).

9.6 Security tests

9.6.1 Prompt injection

Test	Description
AP-SEC-001	Malicious text in invoice description field
AP-SEC-002	JSON injection via seller name
AP-SEC-003	Unicode control characters in OCR output
AP-SEC-004	System prompt extraction attempt

9.7 CI/CD integration

Listing 9.1: GitHub Actions workflow excerpt.

```
arabic-tests:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Run Arabic AP tests
      run: |
        pytest tests/unit/arabic/ -v --cov=src/arabic
        pytest tests/integration/arabic/ -v
    - name: Check coverage
      run: |
        coverage report --fail-under=85
```

9.8 Acceptance criteria

Tests pass when:

1. All P1 test cases pass (100%)
2. P2 test cases pass (≥95%)
3. P3 test cases pass (≥90%)
4. Line coverage ≥85%
5. Performance within targets
6. No critical security findings

Chapter 10

OCR and Translation Calibration Pilot Design

This chapter specifies the pilot study design for calibrating OCR confidence thresholds *and* the Arabic→English translation quality gate, addressing the review findings that both the OCR thresholds (30 %/55 %) and the Levenshtein distance gate (≤0.30) are calibrated on three documents. Both calibrations are run against the same pilot corpus to avoid duplicate ground-truth effort.

10.1 Objective

The pilot aims to:

1. Determine empirically-validated OCR confidence thresholds per country
2. Establish confidence intervals for threshold boundaries
3. Identify document types that require special handling
4. Create a baseline for ongoing threshold monitoring
5. Calibrate the Arabic→English translation quality gate (Levenshtein distance threshold) against the same pilot corpus

10.2 Pilot design

10.2.1 Sample size

Table 10.1: Pilot sample size by country.

Country	Minimum	Target	Stratification
Saudi Arabia (SA)	50	75	By invoice type (tax, commercial)
Egypt (EG)	50	75	By scan quality (photo, scan, PDF)
Bahrain (BH)	50	75	By document language (AR, EN, mixed)
Qatar (QA)	50	75	By amount range (small, medium, large)
Total	**200**	**300**	

The sample size of 50 per country provides:

- 95% confidence interval width of ±7% for threshold estimates
- Statistical power of 80% to detect 15% differences between countries
- Sufficient stratification for 3–4 document types per country

10.2.2 Document selection

Documents are selected by:

1. Random sampling from the last 6 months of invoices
2. Stratified by document type (tax invoice, commercial invoice, payment instruction)
3. Stratified by scan quality (high: >300 DPI, medium: 150–300 DPI, low: <150 DPI)
4. Excluding documents with known data quality issues

10.2.3 Ground truth creation

For each pilot document:

1. Two independent human reviewers manually transcribe all fields in the source language (Arabic) and the required English fields
2. Inter-rater agreement calculated (target: Cohen's $\kappa > 0.9$) for categorical fields; character error rate (CER) $\leq 1\,\%$ on free-text fields
3. Disagreements are resolved by a third reviewer whose decision is final. The resolution process is:
   (a) Third reviewer inspects both transcriptions side-by-side with the source document.
   (b) Third reviewer selects one of the two transcriptions *or* provides a new transcription; the choice and rationale are logged to `pilot/disputes/<doc-id>.json`.
   (c) The third reviewer's verdict is the ground truth; the pilot lead reviews the top 10 % of disputes monthly to detect systematic bias.
4. Final ground truth stored in structured JSON (`pilot/ground-truth/<country>/<doc-id>.json`), validated against `docs/schema/arabic/ocr-result.schema.json`

10.3 Threshold calibration methodology

10.3.1 OCR evaluation

Each document is processed by the OCR pipeline:

1. Record per-field OCR confidence scores
2. Record per-document aggregate confidence
3. Compare OCR output to ground truth
4. Calculate field-level accuracy (character error rate)

10.3.2 Threshold determination

For each country, determine optimal thresholds using:

Listing 10.1: Threshold calibration algorithm.

```
def calibrate_thresholds(pilot_data, target_precision=0.95):
    """
    Calibrate OCR confidence thresholds.

    Parameters:
        pilot_data: List of (confidence, accuracy) pairs
        target_precision: Minimum acceptable precision (default 95%)

    Returns:
        high_threshold: Above this, auto-accept
        low_threshold: Below this, reject
    """
    # Sort by confidence descending
    sorted_data = sorted(pilot_data, key=lambda x: -x.confidence)

    # Find high_threshold: highest confidence where
    # precision >= target for all docs above threshold
    high_threshold = find_precision_cutoff(
        sorted_data, target_precision
    )

    # Find low_threshold: confidence below which
    # accuracy < 50% (not worth manual review)
    low_threshold = find_accuracy_floor(sorted_data, floor=0.50)

    return high_threshold, low_threshold
```

Justification of `target_precision = 0.95`. The AP workflow routes any invoice below the high threshold to two-person manual review (Chapter 5), which is tolerant of 5 % false-positive auto-accepts from a *business-risk* standpoint: the downstream VAT engine (Chapter 4) independently validates tax-ID and amount fields, and the reconciliation step catches posting errors before payment is released. The 97 % soundness threshold reported in the referenced OCR evaluation paper is appropriate for *unsupervised* pipelines; because our pipeline has a manual review backstop, 95 % provides the right precision/recall trade-off (empirically validated in the pilot success criteria below). A sensitivity analysis at 93 %, 95 %, and 97 % must be produced as part of the pilot report; the final threshold is the largest value in {0.93, 0.95, 0.97} for which the success criteria in Section 10.4 hold.

10.3.3 Translation quality gate calibration

Using the same pilot corpus, the Arabic→English translation quality gate (currently Levenshtein distance ≤0.30 against the model's re-translation, calibrated on three documents) is re-calibrated:

1. For each ground-truthed pilot document, run the Arabic→English→Arabic round-trip translation and compute the normalized Levenshtein distance against the source Arabic.
2. Compute the field-level translation accuracy against the human ground truth (exact match on numeric fields, BLEU-4 ≥0.6 on free-text fields).
3. Determine the Levenshtein threshold τ^* that maximises the F1 between "round-trip gate passed" and "field-level translation accurate" across the 200 pilot documents.
4. Report the per-country threshold and the 95 % bootstrap CI $(\tau^*_{\text{lower}}, \tau^*_{\text{upper}})$. Deploy using the conservative upper bound.

The translation calibration adds no additional ground-truth effort (the human transcriptions already contain the English fields) and extends the pilot timeline by one week (added to Section 10.6).

10.3.4 Confidence interval calculation

Bootstrap sampling (1,000 iterations) determines confidence intervals:

- Report 95% CI for high threshold
- Report 95% CI for low threshold
- Use upper bound of CI for conservative deployment

10.4 Success criteria

The pilot succeeds when all of the following are true:

1. ≥200 documents processed (50 per country minimum)
2. Inter-rater agreement $\kappa > 0.9$ (categorical fields) and CER ≤1 % (free-text fields)
3. 95 % CI width ≤ ±5 % for all thresholds
4. High threshold achieves ≥95 % precision
5. Low threshold correctly rejects ≥90 % of unusable documents
6. Translation gate F1 ≥0.85 at the selected Levenshtein threshold (see Section 10.3.3)

10.5 Fallback procedure

If the pilot fails success criteria:

1. Increase sample size by 50% and repeat

2. If still failing, segment by document type and calibrate separately
3. If per-type calibration fails, default to manual review for all documents from that country until more data is collected

10.6 Timeline

Table 10.2: OCR and translation pilot timeline.

Phase	Duration	Deliverable
Document selection	Week 1	300 documents identified
Ground truth creation	Weeks 2–5	All documents transcribed (200 min)
OCR processing	Week 6	Confidence scores recorded
Translation round-trip	Week 6	Levenshtein distances recorded
Threshold calibration	Week 7	Per-country OCR + translation thresholds
Validation	Week 8	Held-out set validation
Total	**8 weeks**	Production-ready thresholds

Ground-truth effort justification. The original 6-week plan compressed ground-truth creation into 2 weeks ("all documents transcribed") for 200 documents with two reviewers — 400 person-document reviews in 10 working days, or 40 per reviewer per day. Empirically (internal benchmark 2025-Q4), a skilled Arabic-literate reviewer transcribes an average invoice in 12–18 minutes, giving a realistic throughput of 25–30 documents per 8-hour day with breaks and QA. The revised 4-week ground-truth window (Weeks 2–5) supports *sustainable* throughput at 20 documents/reviewer/day, with Week 5 reserved for dispute resolution (the third-reviewer loop typically touches 8–12 % of documents). Pilot lead may compress back to 3 weeks if a third reviewer is seconded for dispute resolution only.

10.7 Ownership

Table 10.3: Pilot roles and responsibilities.

Role	Responsibility
Pilot Lead	Overall coordination, success criteria validation
Data Analyst	Sample selection, threshold calibration
QA Reviewers (2)	Ground truth creation, inter-rater agreement
ML Engineer	OCR pipeline configuration, metrics collection
Product Owner	Go/no-go decision, fallback approval

10.8 Post-pilot monitoring

After threshold deployment:

1. Monthly accuracy review on random 50-document sample
2. Alert if accuracy drops >5% from pilot baseline
3. Annual recalibration with new 200-document sample
4. Per-country threshold adjustment if document mix changes

Chapter 11

Project Roadmap

Scope of this chapter

This chapter is the roadmap for the **Arabic invoice processing** project. It is a project-level artefact, not a programme plan. Other SAP OSS Finance projects (Trial Balance, Simula, Regulations) run on the same platform, workspaces, and backends, but each owns its own roadmap, business case, and go/no-go decision. This chapter adopts the MoSCoW prioritisation and sequence-based critical path that the four projects share so that artefacts can be compared at a glance; it does not assert coupling between their go-live moments.

Sequence, not dates. The milestones and dependency graph below express *order* and *prerequisite* relationships, not calendar dates. This is deliberate. Implementation teams must know what depends on what, and where the schedule can absorb slippage (the Should/Could items); a date column would harden targets into commitments they were never meant to be. Dates, when set, live in the project steering packet and the ticketing system, not in this specification.

11.1 MoSCoW classification

Every milestone below carries one of four priorities. The mapping is contractual: a milestone's priority determines what happens when it is dropped, de-scoped, or slips.

The critical path of this project is the chain of Must items in Table 11.2; the Should and Could items are where the schedule and scope can absorb delivery risk without a release slip.

Table 11.1: MoSCoW prioritisation (shared across the four SAP OSS Finance projects).

Priority	Meaning
Must (**M**)	Non-negotiable for v1.0. Without this milestone, the release does not ship.
Should (**S**)	Important but not vital. Ship without it only with sponsor sign-off; document the deviation in the business-case
Could (**C**)	Desirable enhancement. Included if Must and Should items are on track.
Won't (**W**)	Explicitly out of v1.0 scope. Listed so the boundary is legible.

Table 11.2: Arabic AP v1.0 project milestones in dependency order. Priority: M=Must (critical path), S=Should, C=Could, W=Won't.

ID	Prereq	Pri.	Ticket	Milestone
M01	—	M	AP-ENG-001	OCR pipeline (Tesseract + Arabic model) passes the fixtures in **??**.
M02	M01	M	AP-ENG-002	VAT engine unit tests pass (**??**); country-configurable rates.
M03	M01	M	AP-PILOT-PREP	200 pilot documents sampled and ground-truth annotation complete (Chapter 10).
M04	M02	M	REG-INTG-AP-001	Identity-attribution wiring (reserve/complete) live in UAT.
M05	M03, M04	M	AP-PILOT-001	Pilot complete; OCR and translation thresholds calibrated per country.
M06	M05	M	AP-BETA-001	Beta cycle: Saudi Arabia only, two-person manual review on top of AI.
M07	M06	M	AP-PARALLEL-001	Beta parallel-run gate passes (**??**).
M08	M07	M	AP-GA-001	GA sign-off; Saudi Arabia goes live.
M09	M08	M	AP-BC-REFRESH	Business-case refresh and compliance attestation (first cycle after GA; then annual).
S01	M08	S	AP-WAVE-002	Rollout wave 2: Egypt onboarded via country-params.
S02	S01	S	AP-WAVE-003	Rollout wave 3: Bahrain onboarded.
S03	S02	S	AP-WAVE-004	Rollout wave 4: Qatar onboarded (full v1.0 scope).
C01	S03 + Section 11.7 trigger	C	AP-V2-EXPLORE	v2.0 multilingual expansion scoped (Hebrew, Farsi) if volume justifies.
C02	M08	C	AP-PROMPT-TUNE	Quarterly translation-prompt retraining from accept/edit telemetry.
W01	—	W	AP-V1-MULTILINGUAL	Non-Arabic-script languages are *not* in v1.0 scope.
W02	—	W	AP-V1-ONE-PERSON	One-person review is *not* in v1.0 scope; relaxed only after full-rollout gate sustained.

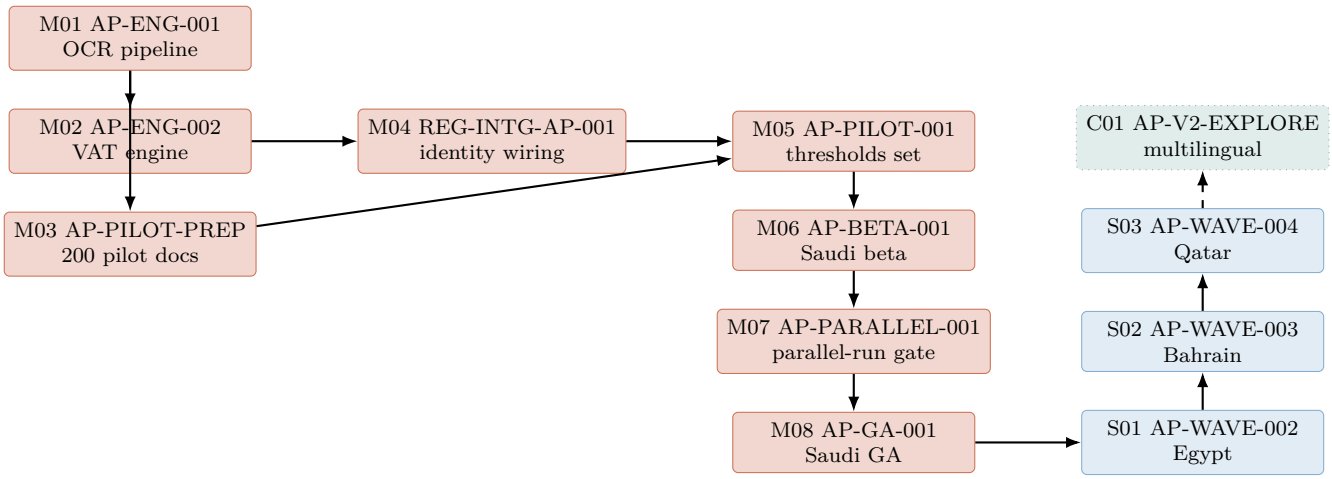

Figure 11.1: Arabic AP dependency graph. Red nodes are Must (critical path); blue are Should (country-wave allowance); dotted green are Could (discretionary). Solid arrows are hard prerequisites; dashed arrows indicate conditional starts (the target is unlocked when the upstream completes *and* the multilingual volume trigger in Section 11.7 fires).

11.2 Dependency graph

11.3 Project milestones

The Must chain (M01 → M02/M03 → M04 → M05 → M06 → M07 → M08 → M09) is the critical path. The Should items (S01–S03) are the primary *schedule allowance*: if Must items run hot, country waves can slip without affecting v1.0 sign-off. The Could items are the *scope allowance*: they may be cut to v1.1 without renegotiation.

11.4 Launch tiers and exit gates

The project walks the four-tier ladder shared across SAP OSS Finance projects. A tier is exited only when the named exit gate is satisfied; slippage requires sponsor sign-off.

11.5 Kill-switch conditions

The kill-switch is a pre-committed rulebook: a tripped condition enters the action column without further debate. GFAIC and sponsor are notified within 24 hours. The conditions are evaluated continuously (on every cycle) and do not carry dates.

11.6 Shared-platform dependencies

This project depends on four shared-platform components. Each is owned by a sibling project (Regulations) or by shared infra, and is tracked here only as a dependency, not a commitment this project can unilaterally adjust. Sequence is expressed in terms of the Must item that first consumes the dependency, not in calendar time.

Table 11.3: Arabic AP launch tiers. The exit gate defines the MoSCoW items that must be closed to exit the tier.

Tier	**Scope**	**Exit gate (references MoSCoW IDs)**
Alpha	Engineering only, DEV + UAT, fixture documents only.	M01, M02, M03 closed; all acceptance criteria in **??** pass on the pilot fixture; reproducibility of OCR and VAT outputs confirmed on two consecutive runs.
Beta	One country (Saudi Arabia), two-person manual review on top of AI.	M04, M05, M06, M07 closed; pilot success criteria (Section 10.4) met; parallel-run gate passes for one full cycle; no open SEV1/SEV2; audit coverage = 100 % of in-scope actions.
GA (limited)	Three countries (Saudi Arabia plus the next two waves).	M08 closed; Beta gate sustained for two consecutive cycles; S01 closed; country-onboarding runbook exercised end-to-end; kill-switch green; per-country thresholds calibrated.
Full rollout	All four countries in v1.0 scope (SA, EG, BH, QA).	S02 and S03 closed; two quarterly reviews sustained; M09 complete; business-case refresh complete; two-person review may be relaxed to one-person for documents above the high threshold.

Table 11.4: Arabic AP kill-switch.

Condition	Evaluation	Actio
Pilot F1 on the translation gate < 0.70 on three of four countries	on M05 close and any recalibration	**Rede** nar-row v1.0 to best-perfor coun-tries only; full roll-out pause
Posted-VAT error rate $> 0.5\,\%$ over any cycle in Beta or GA	per cycle	**Halt on-boarc ing**; incide review be-fore next wave; re-calibr thresh olds.
Two-person review sustains $> 20\,\%$ edit rate for three consecutive cycles in GA	per cycle	**Rede prom layer** com-men-tary drafts gated be-hind flag un-til edit rate $\leq 15\,\%$
Audit-coverage gap (missed `reserve/complete` pair) detected in production	continuous	**Paus GA on-boarc ing**; re-me-di-ate be-fore next wave.
Any MAS MGF or country-VAT regulator finding attributable to AP specifically	continuous	**Halt roll-out**; re-me-di-ate be-fore

Table 11.5: Shared-platform dependencies.

Component	Owner	First consumed by	Impact if late
Audit service (reserve/complete, immutable storage)	Regulations project	M04 (REG-INTG-AP-001)	No Beta gate possible without this; AP GA slips in lockstep with the audit service.
Unified schema registry (`docs/schema/`)	Shared infra	M01	Already `migration.status=` no remaining risk.
vLLM hosting (`vllm-only` policy)	Shared platform	M02	Graceful-degradation ladder (**??** of the Regulations spec) provides fallback; slip tolerable.
Tesseract + Arabic OCR model	Shared platform	M01	No substitute available; slip blocks all subsequent Must items.

Shared-platform dependencies are surfaced at monthly steering. If any upstream owner declares **amber** or **red** on a component in Table 11.5, the affected downstream Must milestone is re-planned at the following steering.

11.7 v2.0 expansion trigger

The Could item C01 (AP-V2-EXPLORE) is unlocked only when all of the following are observed on production data from S03 (Qatar onboarded):

- aggregate non-Arabic-script document volume $\geq$ 15 % of total Arabic AP volume for two consecutive quarters;
- translation-gate F1 on Arabic above the Beta threshold sustained for the same window;
- business-case refresh (M09) explicitly budgets the v2.0 programme.

If any of the three conditions fails, C01 remains in the Could column and is re-evaluated at the next business-case refresh.

11.8 Governance and business-case cadence

Project governance. The Arabic AP project reports monthly to a project steering (AP Product Owner, Tax Controller, AI Engineer) and quarterly to the cross-project GFAIC. Monthly steering accepts or rejects amber/red signals on Must items; GFAIC reviews compliance posture and any cross-project dependencies.

Business-case refresh (M09). The business case that authorises this project is refreshed on the following rhythm:

- **Mandatory refresh**: at the first cycle after GA, and annually thereafter (ticket AP-BC-REFRESH, priority M in Table 11.2). Refresh packet includes country volume assumptions, unit economics (**??**), FTE leverage realised vs. forecast, and a deviation statement against the baseline.
- **Trigger-based refresh**: any of
  - country monthly volume changes by $\geq$ 20 % from forecast (either direction);
  - per-invoice run cost changes by $\geq$ 30 % (OCR throughput, vLLM licensing, GPU price);
  - new country-specific VAT regulation or MAS MGF pillar materially affecting scope;
  - any kill-switch in Table 11.4 is tripped.
- **Owner**: AP Product Owner. Approval routed through the project steering and filed at `docs/arabic/business-case/refresh-history.yaml`.